Retrieval-Augmented Generation (RAG)

A Complete Technical Reference: From First Principles to Production-Ready Pipelines

Chandra Prakash Bathula

AI Systems Engineering

December 9, 2025

"A language model that cannot update its knowledge cannot be trusted with your knowledge.
RAG is how we fix that."

Contents

1 Introduction to Retrieval-Augmented Generation **5**
1.1 What is RAG? . . . 5
1.1.1 The Core RAG Workflow . . . 5
1.2 Why RAG Exists: The Problem It Solves . . . 6

2 Large Language Model Limitations **7**
2.1 How LLMs Store Knowledge . . . 7
2.2 The Three Core Limitations of LLMs . . . 7
2.2.1 Limitation 1: Knowledge Cutoff (Static Knowledge) . . . 7
2.2.2 Limitation 2: Context Window Constraints . . . 8
2.2.3 Limitation 3: Hallucination . . . 8
2.3 How RAG Directly Addresses These Limitations . . . 9

3 System-Level Architecture of RAG **11**
3.1 Two Phases of RAG . . . 11
3.1.1 Phase 1: Offline Indexing (Document Ingestion) . . . 11
3.1.2 Phase 2: Online Query Serving (Retrieval + Generation) . . . 11
3.2 Naive Prompting vs RAG . . . 12
3.3 Typical Use Cases . . . 12

4 Data Ingestion and Preprocessing **14**
4.1 Why Ingestion Quality Defines RAG Quality . . . 14
4.2 Document Types and Their Challenges . . . 14
4.3 Ingestion Tools . . . 14
4.3.1 PyMuPDF (fitz) . . . 14
4.3.2 OCR for Scanned Documents . . . 15
4.3.3 Table Extraction: Docling and Camelot . . . 15
4.4 Preprocessing Statistics . . . 15

5 Chunking: The Heart of RAG **16**
5.1 What is Chunking and Why Does It Matter? . . . 16
5.2 Why Chunking Is Non-Trivial . . . 16
5.3 Key Factors Influencing Chunk Design . . . 17

6 Chunking Strategies in Depth **18**
6.1 Overview of the Five Strategies . . . 18
6.2 Strategy 1: Fixed-Size Chunking . . . 18
6.2.1 Mechanism . . . 18
6.2.2 Sliding Window with Overlap . . . 18
6.2.3 Advantages and Disadvantages . . . 19
6.2.4 Best Used For . . . 19
6.3 Strategy 2: Semantic Chunking . . . 20
6.3.1 Core Idea . . . 20
6.3.2 Algorithm . . . 20

6.3.3 Similarity Threshold Mechanics . . . 20
6.3.4 Advantages and Disadvantages . . . 21
6.4 Strategy 3: Recursive Chunking . . . 21
6.4.1 Core Idea . . . 21
6.4.2 Hierarchy of Split Points . . . 21
6.4.3 Algorithm Steps . . . 21
6.4.4 Advantages and Disadvantages . . . 22
6.5 Strategy 4: Document Structure-Based Chunking . . . 22
6.5.1 Core Idea . . . 22
6.5.2 Sources of Structure . . . 22
6.5.3 Advantages and Disadvantages . . . 23
6.6 Strategy 5: LLM-Based Chunking . . . 23
6.6.1 Core Idea . . . 23
6.6.2 Implementation Approaches . . . 23
6.6.3 Advantages and Disadvantages . . . 24
6.7 Comparison and Selection Guide . . . 24

7 Embeddings: Representing Text as Vectors 25
7.1 From Text to Numbers . . . 25
7.2 Why Embeddings Enable Semantic Search . . . 25
7.3 Levels of Embedding Granularity . . . 26
7.4 Generation 1: Static Word Embeddings . . . 26
7.4.1 Word2Vec (2013) . . . 26
7.4.2 GloVe (Global Vectors, 2014) . . . 26
7.4.3 Critical Limitations for RAG . . . 27
7.5 Generation 2: Contextual Embeddings via Transformers . . . 27
7.5.1 The Transformer Revolution . . . 27
7.5.2 BERT and Its Derivatives . . . 27
7.6 Generation 3: Sentence Transformers for RAG . . . 28
7.6.1 SBERT (Sentence-BERT, 2019) . . . 28
7.6.2 E5 Family (2022–2024) . . . 28
7.6.3 BGE (Beijing General Embedding, 2023) . . . 29
7.6.4 Instructor Embeddings (2023) . . . 29
7.7 Generation 4: Proprietary Embedding APIs . . . 29
7.7.1 OpenAI Embeddings . . . 29
7.7.2 Cohere Embed . . . 29
7.7.3 Voyage AI . . . 29
7.8 Embedding Model Comparison . . . 30
7.9 The Chunking–Embedding Coupling . . . 30
7.10 Embedding Selection Guidelines . . . 31

8 Vector Databases and Similarity Search 32
8.1 The Scalability Problem . . . 32
8.2 Approximate Nearest Neighbor (ANN) Search . . . 32
8.3 HNSW: Hierarchical Navigable Small World Graphs . . . 32
8.3.1 Graph Construction . . . 33
8.3.2 Retrieval Process . . . 33
8.3.3 Key Parameters . . . 33
8.4 IVF: Inverted File Index . . . 33
8.4.1 Index Construction . . . 33
8.4.2 Retrieval . . . 33
8.5 Product Quantization (PQ) . . . 34

8.6 Vector Database Comparison . . . 34
8.7 Metadata Filtering and Hybrid Search . . . 34

9 The End-to-End RAG Pipeline 36
9.1 Complete System Architecture . . . 36
9.2 Query Embedding and Retrieval . . . 36
9.3 Re-Ranking . . . 37
9.4 Prompt Construction . . . 37
9.5 Key Design Trade-Offs . . . 38

10 Advanced RAG Techniques 39
10.1 Query Transformations . . . 39
10.1.1 Query Expansion . . . 39
10.1.2 Sub-Query Decomposition . . . 39
10.2 Indexing Strategies . . . 39
10.2.1 Parent-Child Chunking . . . 39
10.2.2 Summary Indexing . . . 40
10.2.3 Knowledge Graph Augmentation . . . 40
10.3 Evaluation Metrics . . . 40
10.3.1 Core Offline Metrics . . . 40
10.3.2 Retrieval Evaluation . . . 41
10.3.3 Generation and Grounding Evaluation . . . 41
10.3.4 Building an Evaluation Set . . . 41
10.3.5 Offline vs Online Evaluation . . . 42
10.3.6 A Practical Evaluation Loop . . . 42
10.3.7 What to Track During Error Analysis . . . 42

11 Conclusion and Future Directions 43
11.1 What We Have Covered . . . 43
11.2 The Core Design Philosophy . . . 43
11.3 Key Principles for Building Production RAG Systems . . . 44
11.4 Future Directions . . . 44

Chapter 1

Introduction to Retrieval-Augmented Generation

1.1 What is RAG?

Definition: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is an AI system design pattern that *dynamically fetches* relevant information from an external knowledge base at query time, and injects that information into a language model's prompt before generation — producing answers that are both **grounded** and **up-to-date**.

RAG bridges two previously separate disciplines:

- **Information Retrieval (IR)**: Finding relevant documents for a given query.
- **Natural Language Generation (NLG)**: Producing fluent, coherent text from a prompt.

Instead of relying solely on what a model learned during training (its *parametric knowledge*), RAG augments the model with *non-parametric knowledge* — external documents retrieved on demand.

1.1.1 The Core RAG Workflow

At a high level, RAG works as a five-step pipeline:

RAG Query Pipeline

1. **User Query** — The user submits a natural language question.
2. **Query Embedding** — The question is transformed into a numerical vector.
3. **Retrieval** — The vector database finds the most similar document chunks.
4. **Augmentation** — Retrieved chunks are appended to the prompt.
5. **Generation** — The LLM generates an answer *grounded* in the retrieved context.

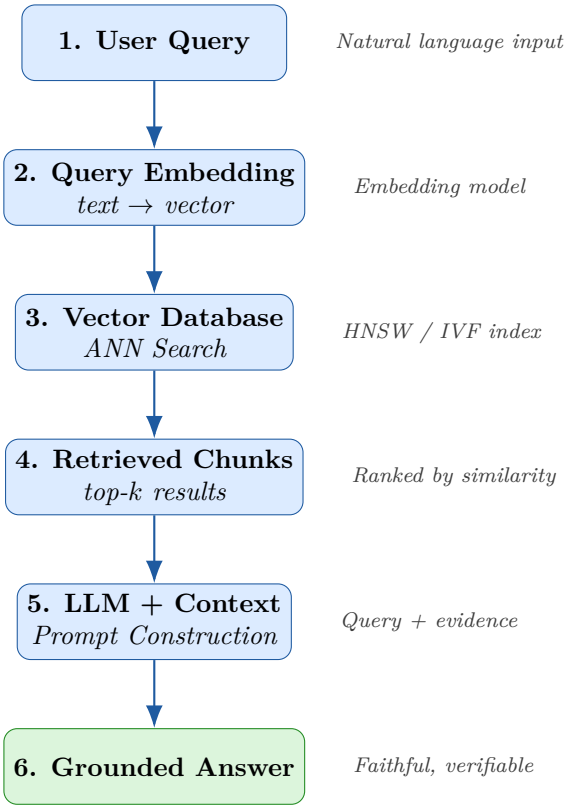

Figure 1.1: End-to-end RAG query pipeline showing data flow from user input to grounded answer.

1.2 Why RAG Exists: The Problem It Solves

To truly appreciate RAG, we must first understand the deep structural limitations of large language models.

Chapter 2

Large Language Model Limitations

2.1 How LLMs Store Knowledge

Large language models learn by predicting the next token across trillions of tokens of text. During this process, they encode statistical patterns, facts, relationships, and reasoning procedures into billions of floating-point weights — their **parameters**.

This type of knowledge is called **parametric knowledge**: it is baked into the model, fixed after training, and retrieved implicitly during inference.

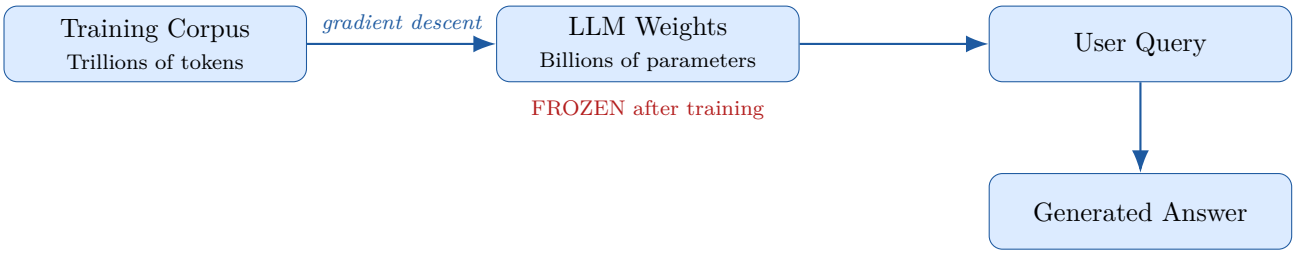

Figure 2.1: LLMs compress training data into fixed model weights. These weights cannot be updated at inference time.

2.2 The Three Core Limitations of LLMs

2.2.1 Limitation 1: Knowledge Cutoff (Static Knowledge)

Every LLM has a **training cutoff date** — a point in time after which it has no knowledge of world events, software updates, regulatory changes, or scientific discoveries.

Problem: Stale Knowledge

If you ask a model trained through mid-2023 about events in late 2024, it will not simply say "I don't know." It is more likely to *generate a plausible-sounding but fabricated answer* based on patterns from its training data.

Examples where static knowledge fails critically:

- Medical treatment guidelines updated quarterly
- Legal statutes amended by legislature
- Software library APIs that changed versions
- Company-internal policies and procedures
- Stock prices, financial reports, product inventories

Retraining is not a solution: Fine-tuning or retraining an LLM on new data costs tens of thousands to millions of dollars and takes weeks to months. It also risks *catastrophic forgetting* of prior knowledge.

2.2.2 Limitation 2: Context Window Constraints

LLMs can only process a fixed number of tokens at once. This limit — the **context window** — defines the maximum amount of information visible to the model during a single inference call.

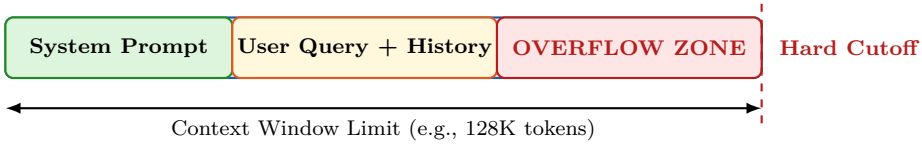

Figure 2.2: The context window has a hard token limit. Content beyond it is truncated or never seen by the model.

Context Window Reality

Even with 128K token context windows, a single enterprise knowledge base, legal document repository, or codebase can contain **billions** of tokens. Cramming everything in is physically impossible.

What Happens When Context Overflows?

- **Truncation**: Oldest or lowest-priority tokens are dropped silently.
- **Lost-in-the-middle effect**: LLMs pay disproportionate attention to the beginning and end of context, often "forgetting" information in the middle.
- **Latency and cost explosion**: Each token in context costs compute money. Sending 128K tokens per query is expensive and slow.

2.2.3 Limitation 3: Hallucination

Definition: Hallucination

Hallucination occurs when an LLM generates text that is *fluent and confident* but *factually incorrect, fabricated, or inconsistent with reality.* The model does not "know" it is wrong — it is simply continuing a statistically plausible token sequence.

Why Hallucination Happens

LLMs are trained to predict the next token. They optimize for *linguistic plausibility*, not *factual accuracy.* When asked about something not well-represented in training data, the model cannot say "I don't know with certainty" — it continues generating based on statistical patterns.

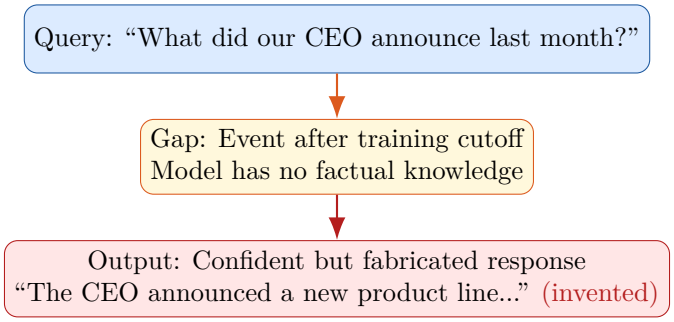

Figure 2.3: Hallucination pipeline: knowledge gap leads to fabricated confident output.

Categories of Hallucination

Type	Description	Example
Factual Fabrication	Inventing facts that don't exist	Citing a paper that was never written
Temporal Hallucination	Applying outdated facts to present	Stating an old CEO is still in position
Attribution Error	Wrong source/author for real content	Misattributing a quote
Intrinsic Inconsistency	Contradicting itself within a response	Saying X then saying not-X
Over-Generalization	Applying a specific fact too broadly	Saying all drugs have a specific property

2.3 How RAG Directly Addresses These Limitations

LLM Limitation	Without RAG	With RAG
Static Knowledge	Model relies on training data only	Fresh documents retrieved at query time
Context Window	Entire corpus cannot fit	Only relevant chunks retrieved
Hallucination	Model fabricates missing facts	Answer grounded in retrieved evidence
Domain Specificity	No enterprise-specific knowledge	Domain documents indexed and retrieved
Verifiability	No way to audit model's reasoning	Source chunks can be shown to user

The RAG Advantage

RAG does **not** retrain the model. It **injects** relevant evidence at query time. This means:

- Knowledge can be updated by simply re-indexing documents.
- The LLM reasons over provided evidence rather than relying on memory.

- Answers can be verified against the retrieved source material.

Chapter 3

System-Level Architecture of RAG

3.1 Two Phases of RAG

RAG consists of two distinct operational phases:

3.1.1 Phase 1: Offline Indexing (Document Ingestion)

Before any query is served, documents must be preprocessed and indexed. This is the **build-time** phase:

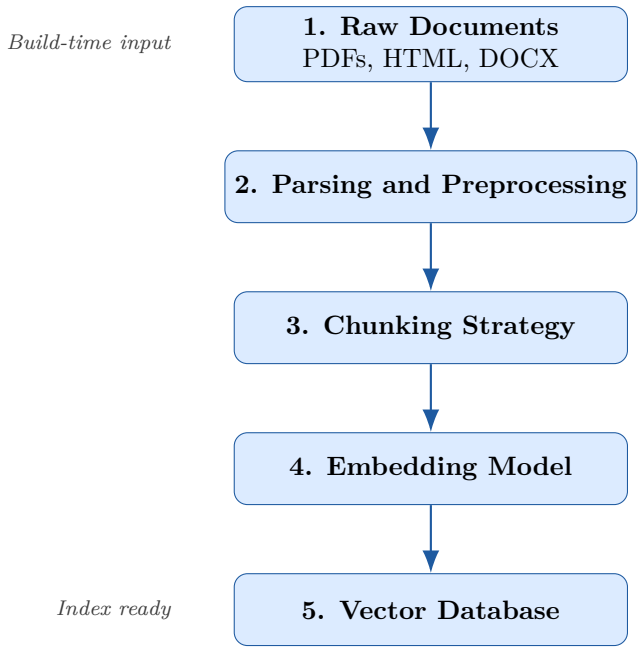

Figure 3.1: Offline indexing pipeline: documents are parsed, chunked, embedded, and stored before any user query is served.

3.1.2 Phase 2: Online Query Serving (Retrieval + Generation)

At inference time, for each user query:

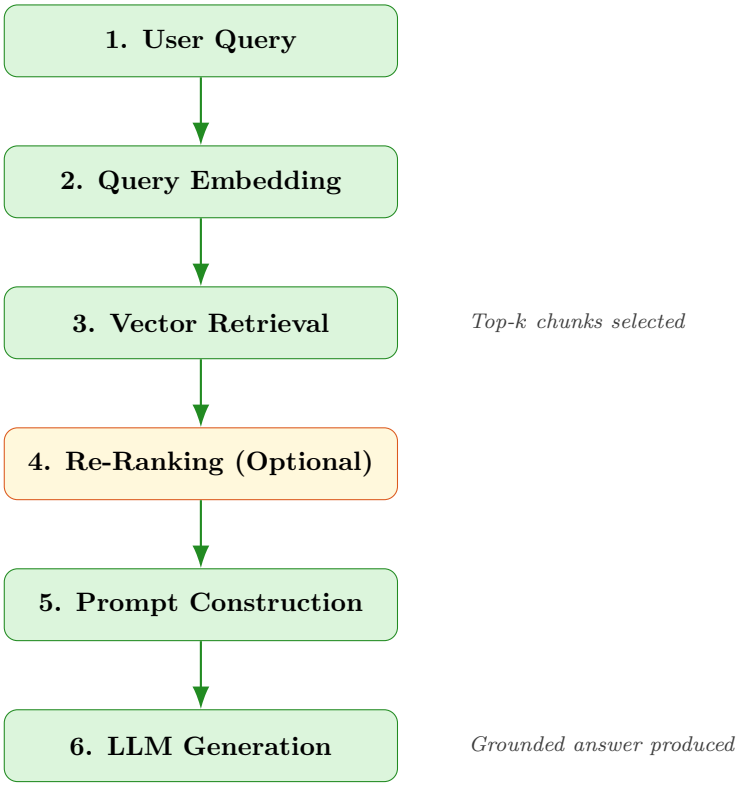

Figure 3.2: Online serving pipeline: the query is embedded, matched to indexed chunks, optionally re-ranked, and then passed to the LLM for grounded generation.

3.2 Naive Prompting vs RAG

Why Naive Prompting Fails

A naive approach inserts entire documents directly into the prompt. This fails because:

1. **Token limits**: Most documents exceed context window sizes.
2. **Cost**: Sending 100K tokens costs roughly 50–100× more than sending 2K tokens.
3. **Attention dilution**: LLMs struggle to focus on relevant facts within very long prompts.
4. **Hallucination**: Important context that doesn't fit is omitted, causing model to invent.

3.3 Typical Use Cases

RAG systems are deployed across industries for:

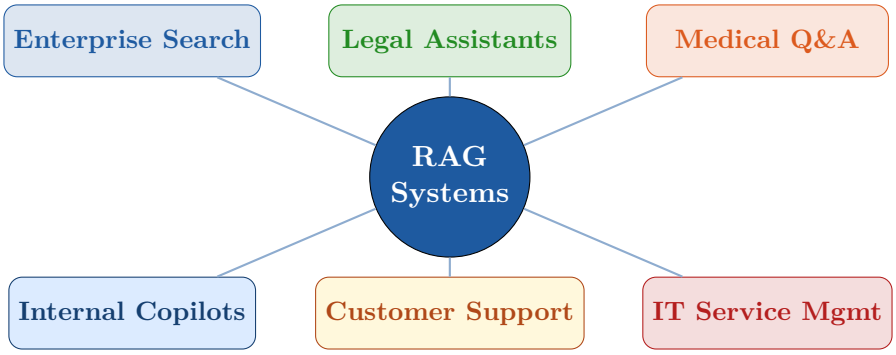

Figure 3.3: Common deployment domains for RAG systems across search, support, operations, and domain-specific assistance.

Chapter 4

Data Ingestion and Preprocessing

4.1 Why Ingestion Quality Defines RAG Quality

The RAG pipeline is only as good as its data ingestion. Garbage in, garbage out. If documents are parsed incorrectly, chunked poorly, or embedded inconsistently, the best retrieval algorithms will still return irrelevant results.

Ingestion Determines Everything Downstream

Every downstream component — chunking, embedding, retrieval, generation — operates on the output of ingestion. Errors compound: a parsing failure cannot be corrected by a smarter retriever.

4.2 Document Types and Their Challenges

Document Type	Characteristics	*Challenge*
Digital text PDF	Machine-readable text layer	*Unicode issues, column layouts*
Scanned PDF	Image pixels, no text layer	*Requires OCR; quality varies*
Hybrid PDF	Mix of text, tables, images	*Needs multi-modal extraction*
HTML / Web	Embedded structure + CSS noise	*Strip boilerplate, keep semantic content*
DOCX / Office	Rich formatting, tracked changes	*Handle styles and inline media*
Structured data	JSON, CSV, XML, databases	*Convert to natural language or handle numerically*

4.3 Ingestion Tools

4.3.1 PyMuPDF (fitz)

Used for efficient extraction of:

- Text per page with layout coordinates
- Embedded images with bounding boxes

- Page-level metadata (author, creation date, title)

4.3.2 OCR for Scanned Documents

For scanned or handwritten documents, Optical Character Recognition (OCR) tools recover text:

- **Tesseract**: Open-source, widely used, supports 100+ languages
- **AWS Textract / Google Document AI**: Cloud-based, high accuracy for forms and tables
- **PaddleOCR**: Strong multilingual and layout-aware OCR

4.3.3 Table Extraction: Docling and Camelot

Tabular content requires special handling. Raw text extraction destroys row/column relationships.

- **Docling**: Converts tables to structured JSON preserving schema — ideal for RAG
- **Camelot / Tabula**: PDF table parsing with lattice and stream modes

4.4 Preprocessing Statistics

Before chunking, compute document statistics to inform design decisions:

Statistic	Purpose	*Informs*
Page count	Estimate total volume	*Chunk budget per page*
Character count	Raw document size	*Memory and storage estimates*
Word count	Human-readable volume	*Expected number of chunks*
Sentence count	Structure density	*Semantic chunking granularity*
Token count	Embedding model compatibility	*Chunk size and overlap settings*
Average sentence length	Text complexity	*Whether to use sentence vs paragraph units*

Chapter 5

Chunking: The Heart of RAG

5.1 What is Chunking and Why Does It Matter?

Definition: Chunking

Chunking is the process of dividing large source documents into smaller, discrete, retrievable text units called **chunks**. Each chunk is independently embedded and indexed. At query time, only the most relevant chunks — not entire documents — are retrieved and passed to the LLM.

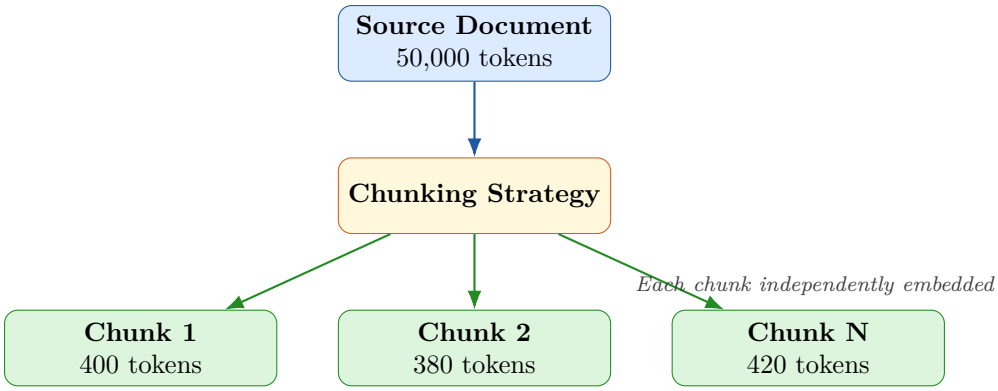

Figure 5.1: Chunking converts one large document into many small retrievable units.

5.2 Why Chunking Is Non-Trivial

Chunking introduces a fundamental tension:

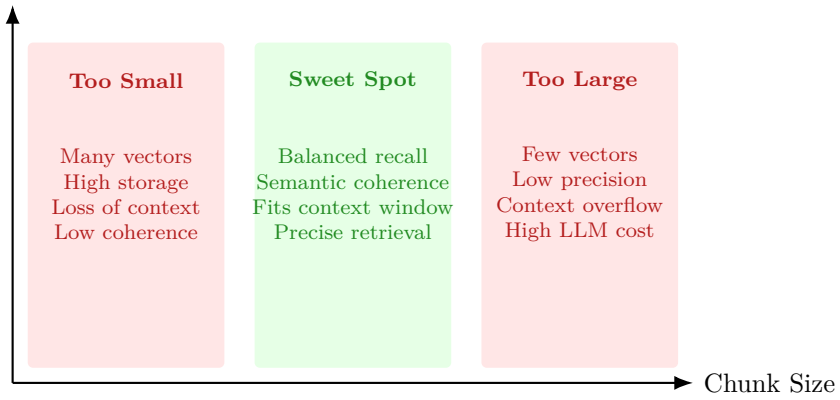

Figure 5.2: The chunk size trade-off: too small loses coherence; too large reduces precision.

5.3 Key Factors Influencing Chunk Design

Chunk Design Decisions

1. **Chunk size**: Number of tokens/characters per chunk
2. **Overlap**: Token overlap between consecutive chunks (typically 10–20%)
3. **Granularity**: Sentence, paragraph, section, or document level
4. **Boundary strategy**: Where to cut (fixed position, sentence end, semantic break)
5. **Metadata**: What context to attach (page number, section title, document ID)

Chapter 6

Chunking Strategies in Depth

6.1 Overview of the Five Strategies

There is no universally optimal chunking strategy. The right approach depends on:

- Document structure (highly structured vs free-form)
- Domain (legal, medical, conversational, technical)
- Query patterns (factoid vs. analytical vs. summarization)
- Computational budget (latency and cost constraints)

Strategy	Core Idea	Complexity	Semantic Quality
Fixed-Size	Split at fixed token count	Low	Low
Semantic	Split at topic shifts	Medium	High
Recursive	Split hierarchically	Medium	Medium-High
Structure-Based	Split at document markers	Low-Medium	High (if structured)
LLM-Based	LLM decides boundaries	Very High	Highest

6.2 Strategy 1: Fixed-Size Chunking

6.2.1 Mechanism

Documents are split into chunks of exactly N tokens (or characters), regardless of content boundaries. This is the simplest possible strategy.

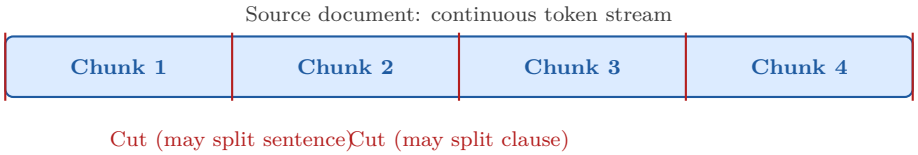

Figure 6.1: Fixed-size chunking splits at uniform intervals, ignoring sentence and paragraph boundaries.

6.2.2 Sliding Window with Overlap

To mitigate hard boundary cuts, consecutive chunks overlap by M tokens:

Sliding Window Math

Given chunk size N and overlap M:

- Chunk 1: tokens 1 to N

- Chunk 2: tokens $N - M + 1$ to $2N - M$
- Chunk k: tokens $(k-1)(N-M) + 1$ to $(k-1)(N-M) + N$

Typical values: $N = 512$ tokens, $M = 50$–100 tokens (10–20% overlap).

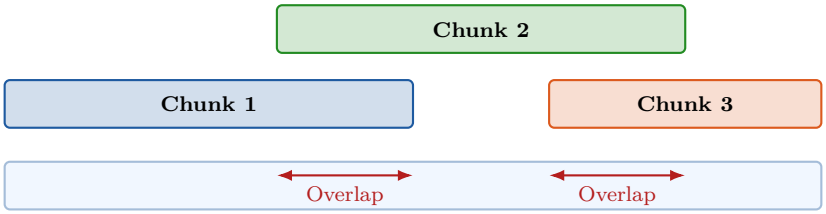

Figure 6.2: Sliding window overlap ensures content at chunk boundaries appears in two consecutive chunks.

6.2.3 Advantages and Disadvantages

Advantages

- Trivially simple to implement
- Predictable chunk sizes
- Compatible with any embedding model
- Extremely fast (no NLP required)
- Scales to billions of tokens

Disadvantages

- Splits sentences and clauses mid-stream
- No awareness of topic shifts
- Overlap adds storage redundancy
- Poor semantic coherence per chunk
- Same size regardless of content density

6.2.4 Best Used For

- Massive, homogeneous corpora (logs, crawled web data)
- Prototyping and baseline systems
- Situations where speed outweighs semantic precision

Example: Legal Contract Analysis

A 200-page legal contract is split into 1000-character blocks with 200-character overlap. A maintenance responsibility clause spanning 1400 characters is split across two chunks. If the retriever returns only chunk one, critical liability qualifiers in chunk two are invisible to the LLM — potentially causing dangerously incorrect legal interpretations.

6.3 Strategy 2: Semantic Chunking

6.3.1 Core Idea

Rather than splitting at fixed positions, semantic chunking groups text by **topical coherence**. The algorithm detects when the subject matter changes and places chunk boundaries at those transition points.

6.3.2 Algorithm

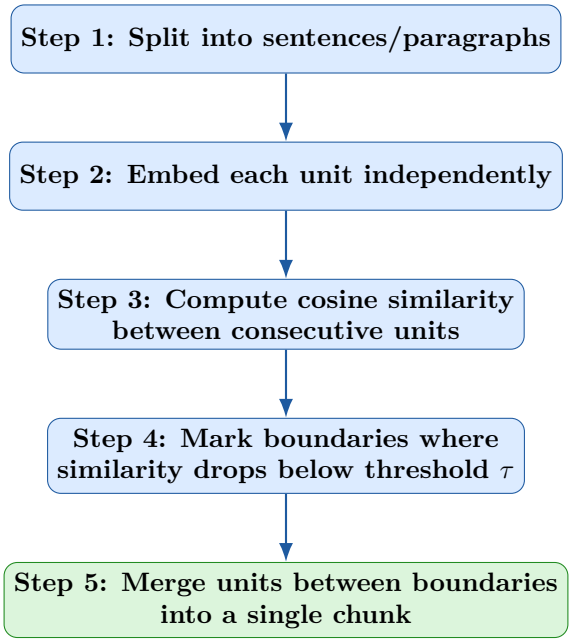

Figure 6.3: Semantic chunking algorithm: embed sentences, detect topic shifts via similarity drops.

6.3.3 Similarity Threshold Mechanics

$$\text{similarity}(s_i, s_{i+1}) = \frac{\mathbf{e}_i \cdot \mathbf{e}_{i+1}}{\|\mathbf{e}_i\| \cdot \|\mathbf{e}_{i+1}\|}$$

- If similarity $\geq \tau$: merge s_{i+1} into current chunk (same topic)
- If similarity $< \tau$: start a new chunk (topic has shifted)

The threshold τ is typically chosen via grid search or percentile-based methods (e.g., place boundary where similarity is below the 25th percentile of all pairwise similarities in the document).

6.3.4 Advantages and Disadvantages

Advantages

- Chunks are semantically coherent
- Adapts to topic length naturally
- Improves retrieval precision significantly
- No arbitrary size assumptions

Disadvantages

- Computationally expensive (N embeddings)
- Threshold τ sensitive and hard to tune
- Chunks can vary greatly in size
- Some chunks may still exceed context limits

Example: History Textbook

In a WWII history textbook, semantic chunking naturally groups together all paragraphs discussing the rise of fascism in Germany into one chunk, and all paragraphs describing the Pacific Theater campaigns into another. A query about Guadalcanal retrieves the Pacific chunk, not the European theater content — exactly the right level of precision.

6.4 Strategy 3: Recursive Chunking

6.4.1 Core Idea

Recursive chunking combines **structural awareness** with **size enforcement**. It splits documents along natural hierarchical boundaries and only subdivides further when chunks are still too large.

6.4.2 Hierarchy of Split Points

The algorithm tries split points in decreasing coarseness order:

Chapter → Section → Subsection → Paragraph → Sentence → Fixed-Size Fallback

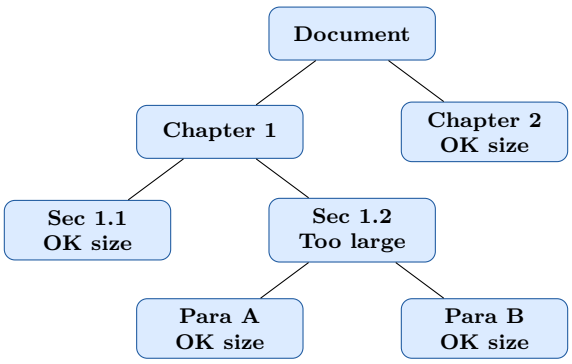

Figure 6.4: Recursive splitting: coarse splits first, then subdivide only oversized units.

6.4.3 Algorithm Steps

1. Split document by highest-level separator (e.g., `\chapter` or '###')

2. For each resulting piece, check if size $\leq N_{\max}$ tokens
3. If yes: keep as a chunk
4. If no: split by next-level separator and repeat from step 2
5. Continue until all pieces satisfy size constraints or the finest separator is reached

6.4.4 Advantages and Disadvantages

Advantages

- Respects document structure
- Guarantees maximum size compliance
- Natural and interpretable boundaries
- Works well with Markdown and structured text

Disadvantages

- Requires knowledge of document separator hierarchy
- Chunk sizes can be uneven
- Complex to implement correctly
- Fails on documents with no consistent structure

Example: Medical Research Papers

A clinical trial paper is split first into sections (Introduction, Methods, Results, Discussion). The Methods section is 4000 tokens — too large. It is recursively split by subsection (Participants, Procedure, Statistical Analysis). Procedure is still 1800 tokens, so it is split again at the paragraph level. This preserves interpretability at every level while fitting embedding model constraints.

6.5 Strategy 4: Document Structure-Based Chunking

6.5.1 Core Idea

This strategy uses **explicit structural markers** embedded by the document author: headings, numbered sections, horizontal rules, HTML tags, or LaTeX commands. Each marked section becomes a chunk.

6.5.2 Sources of Structure

- **Markdown**: `##`, `###` headings create natural boundaries
- **LaTeX**: `\chapter`, `\section`, `\subsection` commands
- **HTML**: `<h1>`–`<h6>` tags, `<section>`, `<article>` elements
- **Word/PDF**: Named styles (Heading 1, Heading 2) in document formatting
- **Numbered lists**: Item 1.1, 1.2 etc. as explicit section identifiers

6.5.3 Advantages and Disadvantages

Advantages

- Aligns perfectly with human interpretation
- Source tracing is trivial ("Section 3.2")
- Simple to implement when structure is reliable
- High coherence for well-structured docs

Disadvantages

- Completely fails on unstructured documents
- Section lengths are wildly inconsistent
- Authors sometimes use headings incorrectly
- Requires document-specific parsing logic

Example: Financial Annual Reports

A company annual report has consistent sections: Executive Summary, Revenue Analysis, Risk Factors, Financial Statements, Notes to Financial Statements. Structure-based chunking creates one chunk per section. A query about "forward-looking risks" retrieves exactly the Risk Factors section without contamination from revenue numbers.

6.6 Strategy 5: LLM-Based Chunking

6.6.1 Core Idea

Instead of using rules or statistical signals, an LLM itself decides where to place chunk boundaries based on semantic understanding of the content. This produces the highest quality boundaries but at the highest computational cost.

6.6.2 Implementation Approaches

1. **Boundary Prompt**: Ask the LLM to output section delimiters.
   "Read this text and output `[BREAK]` wherever a new topic begins."

2. **Topic Classification**: Ask the LLM to label consecutive sentences as "same topic" or "new topic".

3. **Summary-Based**: Ask the LLM to summarize each logical section — the segmentation implied by the summaries defines chunk boundaries.

4. **Iterative Splitting**: Use a smaller LLM for an initial pass, then a larger LLM for refinement.

6.6.3 Advantages and Disadvantages

Advantages

- Highest semantic fidelity
- Handles noisy and unstructured text
- Adapts to domain-specific conventions
- Can handle implicit topic shifts

Disadvantages

- Very expensive (LLM call per chunk)
- Slow — not suitable for large corpora
- Nondeterministic across runs
- Limited by LLM's own context window

Example: Customer Support Transcripts

A customer support chat log contains one conversation that shifts across three issues: delivery delay, damaged packaging, and refund request. An LLM correctly identifies three distinct issue segments and places boundaries at each shift. Rule-based methods would either split mid-sentence or create one massive chunk. This enables precise retrieval — a query about refund policies retrieves only the refund segment.

6.7 Comparison and Selection Guide

Factor	Fixed	Semantic	Recursive	Structure	LLM
Implementation Ease	★★★★★	★★★	★★★	★★★☆	★
Semantic Coherence	★★	★★★★★	★★★☆	★★★☆	★★★★★
Speed / Scalability	★★★★★	★★★	★★★☆	★★★☆	★
Works on Unstructured	★★★★★	★★★★★	★★★	★	★★★★★
Size Controllability	★★★★★	★★	★★★★★	★★	★★★

Hybrid Approach: Best Practice

Production RAG systems frequently combine strategies:

1. **Structure-based** to establish major section boundaries
2. **Recursive** to enforce token limits within sections
3. **Semantic** to verify coherence and split large paragraphs
4. **Fixed-size fallback** for sections that resist all other strategies

Chapter 7

Embeddings: Representing Text as Vectors

7.1 From Text to Numbers

Once documents are chunked, each chunk must be converted into a mathematical representation that supports **semantic similarity search**. This representation is called an **embedding**.

Definition: Embedding

An **embedding** is a mapping from text to a dense, fixed-dimensional vector in $\mathbb{R}^d$:

$$f_\theta : \text{Text} \to \mathbb{R}^d$$

Semantically similar texts are mapped to geometrically close vectors — enabling similarity search by distance computation.

7.2 Why Embeddings Enable Semantic Search

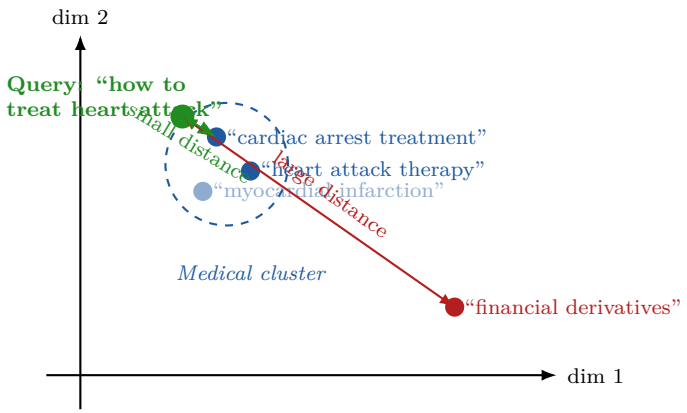

Figure 7.1: In embedding space, semantically similar texts cluster together. Retrieval finds the nearest neighbors to a query vector.

7.3 Levels of Embedding Granularity

Granularity	Description	Use in RAG
Token-level	One vector per subword token	Attention internals only
Word-level	One vector per word (Word2Vec, GloVe)	Rarely used in RAG
Sentence-level	One vector per sentence (SBERT)	Short-document retrieval
Chunk-level	One vector per text chunk	**Primary RAG approach**
Document-level	One vector per full document	Coarse-grained retrieval

Modern RAG systems use **chunk-level embeddings** — each retrieved unit is independently embedded and searched.

7.4 Generation 1: Static Word Embeddings

7.4.1 Word2Vec (2013)

Word2Vec learns dense vector representations by exploiting word co-occurrence patterns in large text corpora. It uses two architectures:

- **CBOW (Continuous Bag of Words)**: Predicts the center word given surrounding context words.
- **Skip-gram**: Predicts surrounding context words given the center word. Better for rare words.

The key result: words used in similar contexts end up close in vector space.

$$\text{vec}(\text{“king”}) - \text{vec}(\text{“man”}) + \text{vec}(\text{“woman”}) \approx \text{vec}(\text{“queen”})$$

7.4.2 GloVe (Global Vectors, 2014)

GloVe builds embeddings from a global word co-occurrence matrix. Unlike Word2Vec's local context windows, GloVe uses corpus-wide statistics:

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

7.4.3 Critical Limitations for RAG

Why Static Embeddings Fail in RAG

1. **Context blindness**: The word "bank" has the same embedding whether it means a financial institution or a riverbank. RAG depends on context-sensitive representations.
2. **No chunk-level encoding**: Word vectors must be manually aggregated (e.g., by averaging) to represent a chunk. This loses sequential relationships.
3. **Out-of-vocabulary**: Words not in training vocabulary have no embedding.
4. **No cross-sentence understanding**: Cannot capture discourse-level meaning across a paragraph.

7.5 Generation 2: Contextual Embeddings via Transformers

7.5.1 The Transformer Revolution

Transformers (Vaswani et al., 2017) introduced **self-attention**, which allows every token to attend to every other token in the input sequence simultaneously:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

where $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), $\mathbf{V}$ (values) are learned linear projections of the token embeddings.

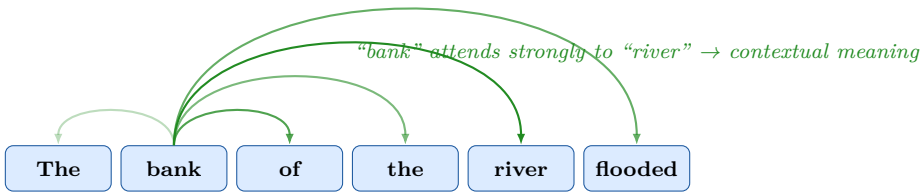

Figure 7.2: Self-attention allows "bank" to attend most strongly to "river," producing a context-specific embedding meaning riverbank rather than financial institution.

This is the fundamental breakthrough enabling contextual embeddings.

7.5.2 BERT and Its Derivatives

BERT (Bidirectional Encoder Representations from Transformers, 2018) introduced:

- **Bidirectional attention**: Unlike GPT which reads left-to-right, BERT attends in both directions simultaneously.
- **Masked Language Modeling (MLM)**: Randomly mask 15% of tokens and train the model to predict them.
- **Next Sentence Prediction (NSP)**: Train on sentence pairs to understand discourse relationships.

The **[CLS]** token's final representation serves as the sentence-level embedding.

Model	Key Innovation	Dimension	Max Tokens
BERT-base	First bidirectional transformer	768	512
BERT-large	Deeper network	1024	512
RoBERTa	Improved BERT training	768	512
ALBERT	Parameter-sharing efficiency	128–4096	512
DeBERTa	Disentangled attention	768	512

7.6 Generation 3: Sentence Transformers for RAG

7.6.1 SBERT (Sentence-BERT, 2019)

Standard BERT is not suitable for semantic similarity search because:

- Computing similarity requires a forward pass on every *pair* of sentences ($O(N^2)$ for a corpus)

CLS token representations are poor out-of-the-box sentence embeddings

SBERT solves this by training with a **siamese network architecture** using contrastive loss:

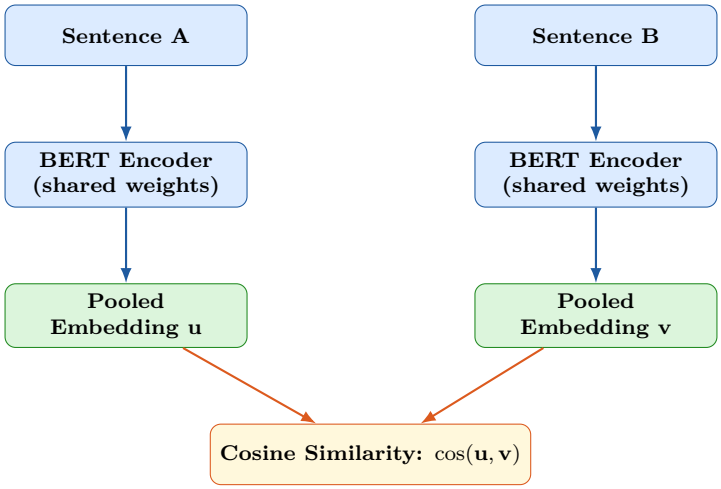

Figure 7.3: SBERT siamese architecture: same encoder processes both sentences independently, then similarity is computed between pooled representations.

7.6.2 E5 Family (2022–2024)

E5 (Embeddings from Bidirectional Encoder Representations) models train on a large mix of supervised and unsupervised data using a **contrastive loss with hard negatives**:

- **E5-small/base/large**: General purpose with excellent quality-to-size ratio
- **E5-mistral-7B**: Uses a 7B-parameter Mistral LLM as encoder — significantly better for long and complex text
- Training signal: query-passage pairs with hard negatives mined from web and NLI data

7.6.3 BGE (Beijing General Embedding, 2023)

BGE models from BAAI (Beijing Academy of AI) achieve top performance on MTEB benchmarks:

- **BGE-small/base/large**: Strong multilingual embeddings
- **BGE-M3**: Supports 100+ languages, handles 8192-token inputs, combines dense + sparse + multi-vector retrieval
- Training: RETROMAE pre-training + contrastive fine-tuning

7.6.4 Instructor Embeddings (2023)

Instructor models accept **task-specific instructions** prepended to the input:

```
"Represent the science document for retrieval: [chunk text]"
"Represent the question for retrieving legal documents: [query]"
```

This instruction conditioning allows a single model to adapt across domains without fine-tuning.

7.7 Generation 4: Proprietary Embedding APIs

7.7.1 OpenAI Embeddings

Model	Dimensions	Max Tokens	Notes
text-embedding-ada-002	1536	8191	Legacy; widely used
text-embedding-3-small	1536	8191	5× cheaper than ada-002
text-embedding-3-large	3072	8191	Highest OpenAI quality

OpenAI's newer models support **Matryoshka Representation Learning (MRL)**, allowing the embedding dimension to be truncated (e.g., to 256) with minimal quality loss — enabling storage-accuracy trade-offs.

7.7.2 Cohere Embed

- **embed-english-v3.0**: 1024 dimensions, 512-token limit, state-of-the-art English retrieval
- **embed-multilingual-v3.0**: 1024 dimensions, 100+ languages
- Supports **input type specification**: `search_document` vs `search_query` — different transformations for indexing and query time

7.7.3 Voyage AI

Anthropic-backed embedding provider with domain-specific models:

- **voyage-large-2**: Best general-purpose quality
- **voyage-code-2**: Optimized for code retrieval (functions, docstrings, comments)
- **voyage-law-2**: Specialized for legal document retrieval
- **voyage-finance-2**: Specialized for financial documents

7.8 Embedding Model Comparison

Model	Dim	Max Tokens	Open Source	Best For
Word2Vec / GloVe	50–300	1 (word)	Yes	Legacy / education
SBERT (all-mpnet)	768	512	Yes	General short text
E5-large-v2	1024	512	Yes	Cost-sensitive production
E5-mistral-7B	4096	32K	Yes	Long, complex text
BGE-M3	1024	8192	Yes	Multilingual, long docs
Instructor-XL	768	512	Yes	Multi-domain with instructions
OpenAI ada-002	1536	8191	No	General production
OpenAI 3-large	3072	8191	No	High-quality production
Cohere embed-v3	1024	512	No	Enterprise multilingual
Voyage-law-2	1024	16000	No	Legal domain
Voyage-code-2	1536	16000	No	Code retrieval

7.9 The Chunking–Embedding Coupling

Chunking and embedding are deeply interdependent:

Critical Design Constraint

Chunk size must be smaller than the embedding model's maximum input length. If a chunk exceeds the model's limit, it will be silently truncated — losing the end of the chunk and corrupting the embedding.

Embedding Model	Recommended Max Chunk Size
SBERT / BERT-based	$\leq$ 450 tokens (512 limit minus query)
OpenAI 3-large	$\leq$ 7500 tokens
BGE-M3	$\leq$ 7500 tokens
E5-mistral-7B	$\leq$ 30K tokens

Additionally, attention-based models embed the *entire chunk sequence jointly*. This means:

- Coherent chunks produce better embeddings (all related tokens attend to each other)
- Fragmented or incoherent chunks produce diluted embeddings
- Poor chunk boundaries degrade both embedding quality and retrieval precision

7.10 Embedding Selection Guidelines

Scenario	**Recommended Model**
On-premise, privacy-critical	**BGE-M3 or E5-large-v2**
Maximum retrieval quality, budget available	**OpenAI text-embedding-3-large**
Multilingual corpus	**BGE-M3 or Cohere multilingual-v3**
Legal documents	**Voyage-law-2**
Code retrieval	**Voyage-code-2**
Long technical documents	**E5-mistral-7B or BGE-M3**
Low-cost experimentation	**E5-small or all-MiniLM-L6-v2**
Education / prototyping	**SBERT or Word2Vec (not production)**

Chapter 8

Vector Databases and Similarity Search

8.1 The Scalability Problem

After embedding N chunks, retrieval requires finding the chunks most similar to a query vector. The naïve approach computes similarity against every stored vector:

$$\text{sim}(q, x_i) = \frac{q \cdot x_i}{\|q\| \cdot \|x_i\|}$$

This is **brute-force nearest neighbor search** with complexity:

$$O(N \cdot d)$$

where d is embedding dimensionality (typically 768–3072) and N is number of chunks.

Corpus Size	Vectors	Brute-Force Latency (1536-dim)
Small demo	10,000	∼2ms
Medium enterprise	1,000,000	∼200ms
Large corpus	100,000,000	∼20 seconds
Web scale	10,000,000,000	∼33 minutes

This makes brute force untenable beyond a few million vectors.

8.2 Approximate Nearest Neighbor (ANN) Search

ANN algorithms trade a small accuracy reduction for massive speed improvements, achieving sub-linear search complexity:

ANN Core Idea

Instead of comparing the query to *all* N vectors, ANN algorithms pre-index vectors into a structure that allows quickly identifying a **small candidate set** likely to contain the true nearest neighbors, then refine within that set.

8.3 HNSW: Hierarchical Navigable Small World Graphs

HNSW is the most widely deployed ANN algorithm in production vector databases.

8.3.1 Graph Construction

HNSW builds a multi-layer graph where each vector is a node:

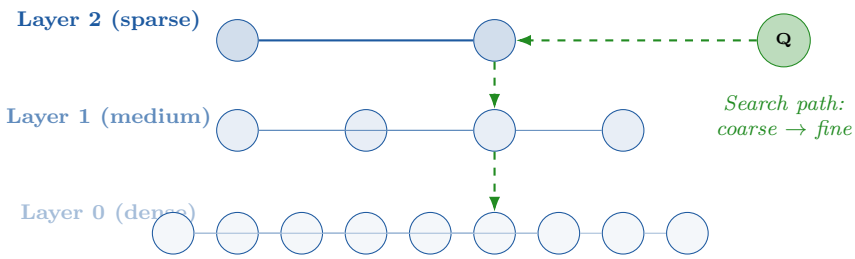

Figure 8.1: HNSW multi-layer graph: query enters at the sparse top layer, navigates greedily toward the nearest node, then descends to denser layers for refinement.

8.3.2 Retrieval Process

1. Enter graph at the highest (sparsest) layer
2. Greedily move to the neighbor closest to the query
3. Descend to the next layer at the current position
4. Repeat until the bottom (densest) layer
5. Collect top-k nearest neighbors at the bottom layer

8.3.3 Key Parameters

Parameter	Effect	Typical Value
M	Number of neighbors per node	16–64
ef_construction	Beam width during index build	100–400
ef_search	Beam width during query	50–200

HNSW achieves $O(\log N)$ query complexity with recall typically above 95%.

8.4 IVF: Inverted File Index

IVF partitions the vector space into clusters using K-Means:

8.4.1 Index Construction

1. Run K-Means with C clusters on all stored embeddings
2. Assign each embedding to its nearest cluster centroid
3. Build an inverted list: centroid → list of embedding IDs

8.4.2 Retrieval

1. Compute query embedding q
2. Find the n_{probe} nearest centroids to q
3. Search all embeddings in those n_{probe} clusters

4. Return top-k by similarity

$$\text{Search space reduced from } N \text{ to approximately } \frac{n_{\text{probe}} \cdot N}{C}$$

Parameter	Guidance
C **(cluster count)**	$\sim\sqrt{N}$ to $4\sqrt{N}$ for balanced recall/speed
$n_{\textbf{probe}}$	Higher = better recall, slower search. Start at $C/10$

8.5 Product Quantization (PQ)

PQ compresses embedding vectors to dramatically reduce memory usage:

1. Split the d-dimensional vector into M equal subspaces of dimension d/M
2. Quantize each subspace independently using its own codebook of k^* centroids
3. Store each vector as M codebook indices (typically 8 bits each)

$$\text{Storage: } d \times 4 \text{ bytes} \rightarrow M \times 1 \text{ byte} \quad (\text{e.g., } 1536 \times 4 = 6144 \text{ bytes} \rightarrow 96 \text{ bytes})$$

PQ is typically used with IVF (IVF-PQ) for billion-scale deployments.

8.6 Vector Database Comparison

Database	Index	Scale	Hosted	Open	Best For
Pinecone	HNSW	Billions	Yes	No	Managed production
ChromaDB	HNSW	Millions	No	Yes	Local dev / prototyping
Weaviate	HNSW	Billions	Both	Yes	Hybrid search + modules
Milvus	HNSW+IVF+PQ	Billions	Both	Yes	High-throughput enterprise
Qdrant	HNSW	Billions	Both	Yes	Filtering + ANN
FAISS	HNSW+IVF+PQ	Billions	Library	Yes	Research / custom builds
pgvector	HNSW+IVF	Millions	No	Yes	Existing PostgreSQL stack

8.7 Metadata Filtering and Hybrid Search

Modern vector databases support filtering on metadata *during* vector search:

Metadata Filter Example

Query: "What is the maintenance procedure?"
Filter: `document_type = "service_manual" AND year >= 2023`
Result: Only vectors from recent service manuals are searched.

Hybrid search combines dense vector retrieval with sparse BM25/TF-IDF keyword matching:

$$\text{score}(d, q) = \alpha \cdot \text{dense_sim}(d, q) + (1 - \alpha) \cdot \text{BM25}(d, q)$$

This captures both semantic similarity and exact keyword matches, significantly improving retrieval for queries with specific terminology.

Chapter 9

The End-to-End RAG Pipeline

9.1 Complete System Architecture

The complete RAG pipeline integrates all previous components:

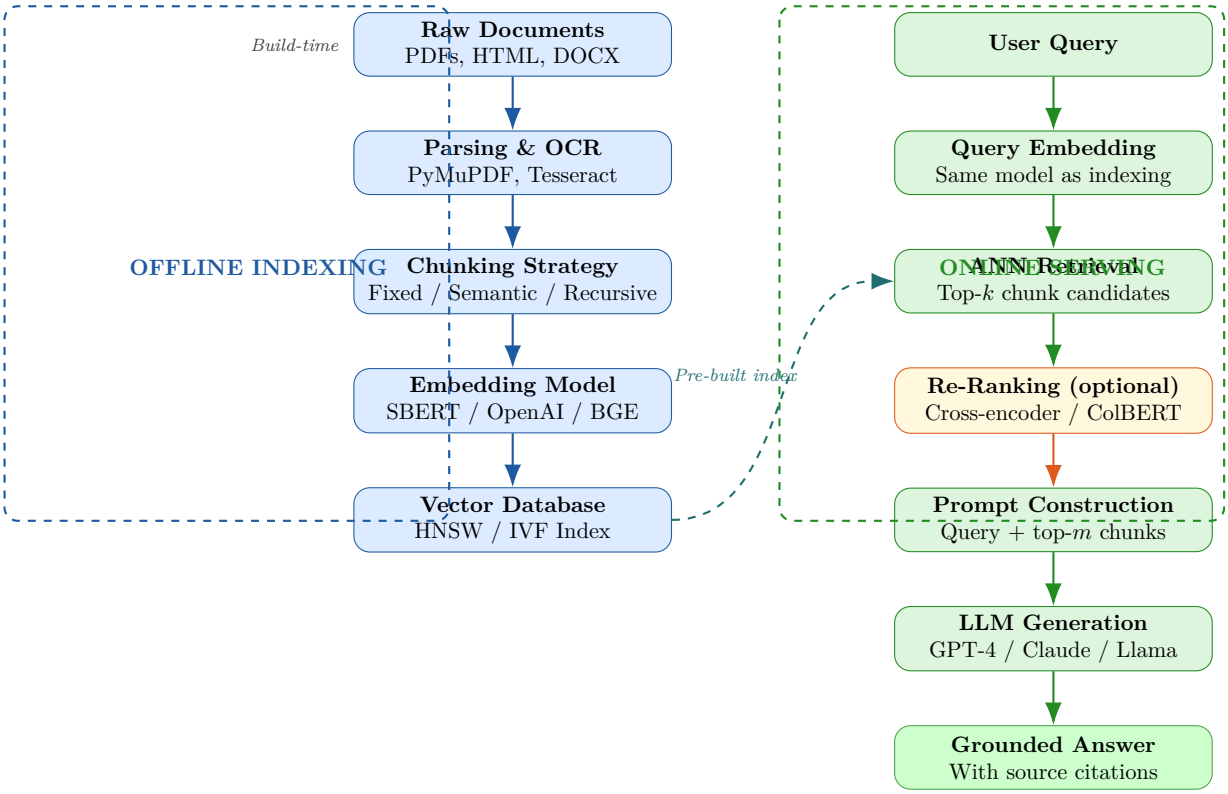

Figure 9.1: Complete RAG system: left pipeline (blue) is the offline indexing phase; right pipeline (green) is the online query serving phase. The vector database bridges both.

9.2 Query Embedding and Retrieval

Given user query q_{text}:

$$q = f_\theta(q_{\text{text}}) \in \mathbb{R}^d$$

Retrieve top-k using ANN search:

$$\{x_{(1)}, x_{(2)}, \ldots, x_{(k)}\} = \underset{x_i \in \text{Index}}{\text{top-k}} \quad \cos(q, x_i)$$

Each result carries its chunk text, metadata (source, page, section), and similarity score.

9.3 Re-Ranking

Vector similarity scores are approximate — the top retrieved chunk by cosine similarity is not always the most relevant to the query. Re-ranking adds a second pass:

- **Cross-encoder**: Takes (query, chunk) as a joint input, producing a relevance score. Far more accurate than bi-encoder similarity but requires one forward pass per candidate.
- **ColBERT**: Late interaction model. Stores token-level embeddings per chunk and computes MaxSim across query tokens at retrieval time — balancing quality and speed.
- **LLM re-ranking**: Ask the LLM to rank candidates by relevance. Most accurate, but highest latency.

$$\text{Re-rank: } \{x_{(1)}, \ldots, x_{(k)}\} \xrightarrow{\text{cross-encoder}} \{x_{[1]}, \ldots, x_{[m]}\}, \quad m \leq k$$

9.4 Prompt Construction

The augmented prompt combines the user query with retrieved context:

RAG Prompt Template

```
You are a helpful, precise assistant.
Answer the QUESTION using ONLY the information in the CONTEXT below.
If the answer cannot be found in the context, say "I don't know."
Cite the source section when making specific claims.

CONTEXT:
[Source: Q3 2024 Product Manual, Section 3.2]
The maintenance interval for pump model XR-7 is every
500 operating hours or 6 months, whichever comes first...

[Source: Q3 2024 Product Manual, Section 4.1]
When replacing the XR-7 seal kit, ensure the system is
fully depressurized before disassembly...

QUESTION: How often should I maintain the XR-7 pump?

ANSWER:
```

Key prompt engineering decisions:

- Include source metadata to enable citation and verification
- Explicitly instruct the model to use *only* the provided context
- Instruct model to say “I don’t know” rather than hallucinate
- Set appropriate length and format expectations

9.5 Key Design Trade-Offs

Decision	Too Low / Small	Too High / Large
Top-k retrieved	Missing relevant context	Noisy, incoherent prompt
Chunk size	Loss of coherence, too many vectors	Imprecise retrieval, high cost
Overlap	Boundary sensitivity	Storage bloat
Re-rank cutoff (m)	Precision loss	Latency increase
Context window usage	Incomplete coverage	High LLM inference cost

Chapter 10

Advanced RAG Techniques

10.1 Query Transformations

Raw user queries are often ambiguous or do not match the vocabulary of source documents. Query transformations improve retrieval:

10.1.1 Query Expansion

The original query is expanded with additional terms or reformulations:

- **HyDE (Hypothetical Document Embeddings)**: Ask the LLM to generate a hypothetical answer to the query, then embed *that* as the search vector. This bridges vocabulary mismatch between query and corpus.
- **Multi-query**: Generate 3–5 diverse reformulations of the query and retrieve for each. Merge and deduplicate results.
- **RAG-Fusion**: Multi-query + Reciprocal Rank Fusion to combine ranked result lists.

10.1.2 Sub-Query Decomposition

Complex questions are decomposed into simpler factoid sub-questions, each answered independently, then synthesized:

Sub-Query Decomposition Example

Original: "How did the 2024 banking crisis affect European vs US interest rate policy?"
Decomposed:

1. What were the key events of the 2024 banking crisis?
2. How did the ECB adjust interest rates in 2024?
3. How did the Federal Reserve adjust interest rates in 2024?
4. What were the stated reasons for each central bank's decisions?

10.2 Indexing Strategies

10.2.1 Parent-Child Chunking

Store **small chunks for precise retrieval** but retrieve **larger parent chunks** for generation context:

- Index: sentence-level chunks (50–100 tokens) for high precision retrieval
- Retrieve: the parent paragraph (400–600 tokens) surrounding the matched sentence
- Result: precise match + rich context for the LLM

10.2.2 Summary Indexing

For each chunk, also index:

- A one-sentence LLM-generated summary of the chunk
- The full section heading path (e.g., "Chapter 3 > Methods > Participants")

Queries match against summaries; the original chunk text is passed to the LLM.

10.2.3 Knowledge Graph Augmentation

Extract entities and relationships from documents and build a knowledge graph alongside the vector index. Complex multi-hop queries traverse the graph; factoid queries use the vector index. This is the basis of **GraphRAG**.

10.3 Evaluation Metrics

A RAG system should never be evaluated only by whether answers *sound good*. Proper evaluation separates failures into at least three layers:

1. **Retrieval quality**: Did the system fetch the right evidence?
2. **Grounding quality**: Did the model stay faithful to that evidence?
3. **Task success**: Did the final answer solve the user's actual problem?

Evaluation Principle

If retrieval fails, generation quality is largely irrelevant. A fluent answer built on missing or weak evidence can still be wrong. This is why RAG evaluation must begin with retrieval, not with stylistic answer quality alone.

10.3.1 Core Offline Metrics

Metric	Description	Tool
Retrieval Precision@k	Fraction of top-k retrieved chunks that are relevant	RAGAS
Recall@k	Fraction of all relevant chunks that appear in top-k	RAGAS
MRR (Mean Reciprocal Rank)	Measures rank of first relevant result	BEIR
Answer Faithfulness	Does the answer use only the retrieved context?	RAGAS
Answer Relevance	Does the answer actually address the query?	RAGAS
Context Recall	Does retrieved context cover all needed information?	RAGAS
Hallucination Rate	Frequency of claims not supported by context	LLM-as-judge

10.3.2 Retrieval Evaluation

Retrieval should be measured independently from generation using a labeled query set. For each query, annotate:

- the truly relevant chunk or chunks,
- acceptable alternate chunks,
- metadata constraints (document type, date, jurisdiction, product, team, etc.).

Then compute:

- **Precision@k**: useful when prompt budget is limited and every retrieved chunk must earn its place,
- **Recall@k**: useful when missing even one critical chunk causes failure,
- **MRR / NDCG**: useful when ranking quality matters, not just set overlap.

Why Retrieval Metrics Matter

A system can achieve high answer fluency while still having poor Recall@k. In that case, the language model is compensating for missing evidence rather than being grounded in it. This is precisely the failure mode RAG is supposed to prevent.

10.3.3 Generation and Grounding Evaluation

Once retrieval is acceptable, evaluate the generated answer itself:

- **Faithfulness**: every factual claim should be traceable to retrieved context,
- **Answer relevance**: the answer should directly address the user question,
- **Completeness**: the answer should cover all major sub-parts of the question,
- **Citation accuracy**: cited sources should actually support the claims they are attached to.

In high-stakes domains, a useful rubric is to score each answer on **correctness**, **support**, and **omissions** separately rather than collapsing everything into one score.

10.3.4 Building an Evaluation Set

A robust evaluation set should contain a balanced mix of query types:

- **Fact lookup**: single-hop, directly answerable from one chunk,
- **Multi-hop synthesis**: requires combining information from multiple chunks,
- **Ambiguous queries**: require clarification or careful ranking,
- **No-answer cases**: the corpus does not contain the answer and the model should abstain,
- **Adversarial queries**: lexical mismatch, distractor documents, or misleading partial evidence.

Good Evaluation Data

The best benchmark queries are not generic. They are drawn from real user behavior, production logs, support tickets, analyst workflows, or domain-specific tasks. Synthetic data can help bootstrap evaluation, but real queries reveal the true failure modes.

10.3.5 Offline vs Online Evaluation

Offline evaluation is necessary, but production systems also need online monitoring:

- click-through on retrieved sources,
- citation open rate,
- user satisfaction or thumbs-up rate,
- task completion rate,
- abstention rate when context is insufficient,
- latency and cost per answered query.

Offline metrics tell you whether the system *can* work; online metrics tell you whether it *does* work for real users under real traffic.

10.3.6 A Practical Evaluation Loop

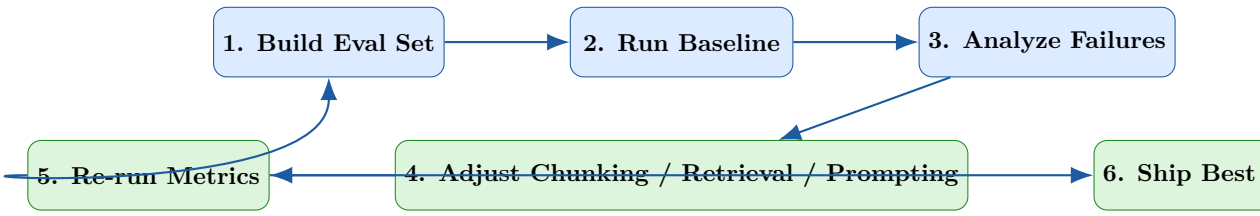

Figure 10.1: Practical RAG evaluation loop: build a benchmark, measure a baseline, inspect failures, iterate on the pipeline, and only then deploy improvements.

10.3.7 What to Track During Error Analysis

When a query fails, classify the root cause explicitly:

- **Chunking failure**: the answer was split across chunks or buried in noisy context,
- **Embedding failure**: semantically relevant text was not mapped near the query,
- **Ranking failure**: relevant chunks were retrieved but scored too low,
- **Prompting failure**: evidence was present but the model ignored or misused it,
- **Knowledge gap**: the answer genuinely does not exist in the corpus.

This classification matters because each failure implies a different fix. Changing the prompt will not solve an embedding failure, and changing the embedding model will not solve a missing-data problem.

Chapter 11

Conclusion and Future Directions

11.1 What We Have Covered

This document has built a complete understanding of Retrieval-Augmented Generation from first principles:

Chapter Summary

1. **LLM Limitations**: Static knowledge, context window constraints, and hallucination — the three problems RAG solves
2. **RAG Architecture**: Two-phase system (offline indexing + online serving) and why it outperforms naive prompting
3. **Data Ingestion**: Document parsing, OCR, table extraction, and statistical profiling
4. **Chunking**: Five strategies (fixed-size, semantic, recursive, structure-based, LLM-based) with trade-offs and domain examples
5. **Embeddings**: From static Word2Vec through transformer-based contextual models to production proprietary APIs
6. **Vector Databases**: ANN algorithms (HNSW, IVF, PQ), metadata filtering, and hybrid search
7. **End-to-End Pipeline**: Query embedding, retrieval, re-ranking, prompt construction, and generation
8. **Advanced Techniques**: Query expansion, parent-child indexing, and evaluation frameworks

11.2 The Core Design Philosophy

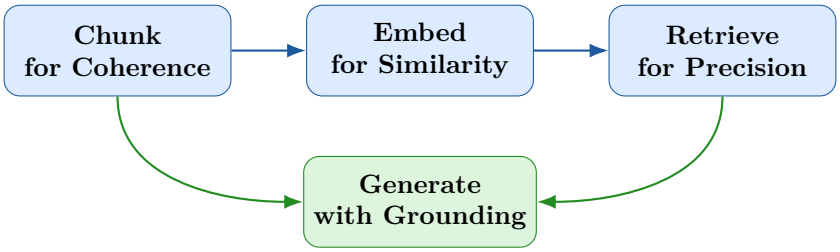

Figure 11.1: The RAG design philosophy: chunking, embedding, and retrieval jointly enable grounded generation.

11.3 Key Principles for Building Production RAG Systems

1. **Iteration is mandatory**: No single configuration is optimal out of the box. Measure retrieval quality with RAGAS or equivalent, then iterate.
2. **Domain matters**: Legal, medical, code, and conversational corpora each have different optimal chunking, embedding, and retrieval configurations.
3. **Chunk coherence beats chunk size**: A semantically coherent 600-token chunk outperforms a perfectly sized but fragmented 400-token chunk.
4. **Match embedding models to chunk size**: Never exceed the embedding model's input limit. Truncated embeddings degrade retrieval silently.
5. **Hybrid search consistently outperforms**: Combining dense semantic retrieval with sparse BM25 keyword retrieval reduces failure modes.
6. **Re-ranking is worth the latency**: Even a lightweight cross-encoder re-ranker substantially improves answer quality.
7. **Ground truth prevents hallucination**: Instruct the LLM to use only retrieved context and to acknowledge when context is insufficient.

11.4 Future Directions

The RAG field is evolving rapidly:

- **Adaptive chunking**: Algorithms that automatically adjust strategy and parameters based on document type and query pattern analysis
- **Longer embedding windows**: Models like E5-Mistral and BGE-M3 already support 32K tokens; 100K+ context embeddings are emerging
- **Multimodal RAG**: Unified retrieval over text, images, tables, audio, and video
- **Agentic RAG**: LLM agents that iteratively refine queries, decide when to retrieve, and chain multiple retrieval steps
- **GraphRAG and structured knowledge**: Combining vector retrieval with knowledge graphs and structured databases for complex multi-hop reasoning
- **Streaming RAG**: Real-time document ingestion and index updates for continuously changing corpora
- **Personalized RAG**: User-level retrieval preferences and relevance feedback loops

Closing Thought

Retrieval-Augmented Generation is not merely a technique for working around LLM limitations — it is a fundamental architectural pattern for building AI systems that remain **accurate**, **trustworthy**, **verifiable**, and **up-to-date** in the face of a continuously changing world. The principles in this document provide a rigorous foundation for building such systems at any scale.